



مراجعة تقنية — البلوكتشين والمخططات

المخطط المفاهيمي · المخطط المنطقي (UML) · مخطط الفئات · اعتماد الأرض على السلسلة

منصة صرح · الرؤية الليبية للاتصالات والتقنية — LVCT · 2026

صّرح — مراجعة تقنية: طبقة البلوكتشين والمخططات (المفاهيمي / المنطقي UML / الفئات)

الغرض من هذه الوثيقة: وثيقة مكتفية بذاتها للمراجع تجمع — في ملف واحد — المخطط المفاهيمي (Conceptual ERD)، والمخطط المنطقي (Logical ERD / UML)، ومخطط الفئات (Class Diagram)، مع شرح كامل ومرجعية الكود لكيفية اعتماد الأرض على البلوكتشين (سكّ رخصة NFT). أنشئت هذه الوثيقة لأن طبقة البلوكتشين وكودها لم تظهر في نسخة سابقة سُلّمت للمراجعة، رغم أنها موجودة بالكامل في المستودع.

٠. ملخص للمراجع (الإجابة المباشرة)

١. كود البلوكتشين موجود وكامل داخل المستودع، وليس ناقصاً. (انظر خريطة الملفات في القسم ١.)

٢. «اعتماد أرض على البلوكتشين» في صّرح يعني: سكّ رخصة ملكية رقمية كـ NFT بمعيّار

ERC-721. تتم العملية في الدالة `LicenseService.FinalApproveAsync`

(`apps/api-dotnet/Workflow/LicenseService.cs`) عند الاعتماد النهائي لعقار حالته `approved`.

٣. طبقة السلسلة مُجرّدة خلف واجهة `IBlockchainService`. التنفيذ الحالي الافتراضي هو **Stub** حتمي (مناسب للتطوير والعرض دون عقدة RPC أو رسوم غاز)، وهو **قابل للاستبدال بتنفيذ Ethereum حقيقي (Nethereum) ** دون تغيير أي سطر في طبقة الخدمات أو قاعدة البيانات.

هذا تصميم متعمّد (Strategy / Ports & Adapters)، وهو موثّق في تعليقات الكود نفسه.

٤. سجل الملكية على السلسلة يُمثّل في جدولين: `property_nfts` (الرابط بين العقار وهويته

على السلسلة) و `ownership_history` (سلسلة حيازة للإضافة فقط — مُنع عليها `UPDATE/DELETE` بمشغّلات قاعدة بيانات). كلاهما عُرّف في الترحيل

`028_property_nfts_ownership_history.sql`.

ملاحظة أمانة علمية: ما هو مُنقذ ويعمل اليوم هو طبقة السلسلة بنمط Stub حتمي + نموذج

البيانات الكامل + خط الإنتاج الكامل (سكّ، نقل، تحقّق). التنفيذ الفعلي على شبكة إيثيريوم حيّة (Sepolia/Mainnet) هو نقطة تبديل موثّقة (تغيير إعداد + إضافة حزمة

١. خريطة كود البلوكتشين (أين يقع كل شيء)

كل ما يخصّ البلوكتشين يقع في مكانين رئيسيين داخل خدمة الـ API (.NET 8):

المسار	النوع	الغرض
apps/api-dotnet/Blockchain/IBlockchainService.cs	واجهة	عقد طبقة الـ rOfAsync + روابط الـ sService
apps/api-dotnet/Blockchain/StubBlockchainService.cs	تنفيذ	بديل حتمي tokenId لمعرّف العن
apps/api-dotnet/Blockchain/StubIpfsService.cs	تنفيذ	بديل IPFS: على القرص (الـ CID = ت)
apps/api-dotnet/Blockchain/PropertyLicenseMetadata.cs	نموذج	مخطط ميتا (OpenSea, GeoJSON)
apps/api-dotnet/Blockchain/BlockchainOptions.cs	إعدادات	قسم chain و rh:Ipfs و ethereum) العقد، Url
apps/api-dotnet/Workflow/LicenseService.cs	خدمة	قلب الاعتماد الرخصة (c)
apps/api-dotnet/Workflow/TransferService.cs	خدمة	نقل ملكية (بيع/إرث/قر)
apps/api-dotnet/Workflow/NftsService.cs	خدمة	قراءة سجلّ (للمدير/المصنّع)
apps/api-dotnet/Controllers/PropertiesController.cs	مسارات	POST d}/final- → approve
apps/api-dotnet/Controllers/NftsController.cs	مسارات	property-transfer
apps/api-dotnet/Controllers/MeController.cs	مسارات	/me/nfts المواطن.

المسار	النوع	الغرض
apps/api-dotnet/Data/Entities/PropertyNft.cs + OwnershipHistory.cs	كيانات	كيانات Core الجدولين.
infra/mssql/migrations/028_property_nfts_ownership_history.sql	ترحيل	إنشاء + ts _history مشغلات الإ
apps/api-dotnet/Program.cs (الأسطر ١٠٨-١٢٥)	تسجيل DI	حقن Service و الإعدادات.
packages/shared-types/src/index.ts	أنواع	أنواع Script (shipEvent)
apps/web/src/app/core/nfts.service.ts + admin/nft- صفحات licences*	واجهة	عرض الرخد المستكشف

٢. كيف تُعتمد أرض على البلوكتشين (التدقق الكامل + الكود)

٢.١ نظرة عامة على المسار

اعتماد الأرض على السلسلة هو **الخطوة الأخيرة** في دورة حياة العقار، بعد المراجعة الفنية والاعتماد. التسلسل الكامل للحالة:

draft → pending → under_review → approved → minted → (transferred)*

السكّ نفسه يحدث عند الانتقال من approved إلى minted ، وينفّذ التالي بالترتيب داخل : LicenseService.FinalApproveAsync

1. التحقق من الصلاحية والنطاق الجغرافي للطالب.
2. **فحص العود (Idempotency):** إن وُجدت رخصة غير فاشلة للعقار → تُعاد كما هي بلا سكّ مكرّر.
3. التأكد أن حالة العقار approved (اعتمدها موظف السجل وصدر الصكّ والاعتماد ب VC).
4. **بوابة أمنية:** رفض السكّ إن كان على العقار نزاع/حجز نشيط
(AssertNoActiveDisputeAsync).

5. بناء الميْتادات (اسم، وصف، سمات، امتداد sarh يضمّ مضلّع GeoJSON ورقم القرار وتجزئة الصكّ).

6. تثبيت الميْتادات على IPFS → الحصول على sha256 + ipfs://<cid> .

7. السكّ على السلسلة عبر chain.MintAsync(...) → إيصال يضمّ tokenId , contractAddress

. network , txHash , blockNumber , ownerAddress

1. الحفظ في قاعدة البيانات ضمن SaveChanges واحدة: صفّ في property_nfts (حالة minted

+ صفّ أول في ownership_history (from=null , reason='initial_mint' .

1. تحديث حالة العقار إلى minted + ختم المُعتمِد وتاريخ الاعتماد النهائي ورقم القرار.

2. إشعار المواطن بأن رخصة عقاره صدرت على البلوكتشين (مع tokenId ورابط المستكشف).

٢.٢ الشروط المسبقة والبوابات الأمنية

```
// LicenseService.FinalApproveAsync - الشروط قبل السكّ
// فحص الغُود: لا نسكّ رمزاً مكرراً لعقار له رمز قائم (1)
var existing = await db.PropertyNfts
    .FirstOrDefaultAsync(n => n.PropertyId == property.Id && n.Status != "failed", ct);
if (existing is not null)
    return await BuildResultAsync(property, existing, ct);

// 2) لا سكّ قبل اعتماد موظف السجل.
if (property.Status != "approved")
    throw SarhException.Conflict("لا يمكن سكّ الرخصة قبل اعتماد موظف السجل");

// بوابة أمنية: لا سكّ لعقار عليه نزاع/حجز نشط (حجز محكمة، رهن، وقف...) (3)
await disputes.AssertNoActiveDisputeAsync(property.Id, ct);
```

٢.٣ بناء ميْتادات الرخصة (المرساة خارج السلسلة)

الميتاداتا تُخزّن خارج السلسلة (IPFS) وتُرسى تجزئتها على السلسلة عبر `tokenURI`. المخطط يتبع عُرف EIP-721 / OpenSea ليقرأه أي محفظة أو مستكشف، مع امتداد `sarh` يحمل البيانات الخاصة بالسجل (المضلع الجغرافي، رقم القرار، تجزئة الصك، رابط التحقق):

```
// LicenseService.BuildMetadata - المقتطف الجوهري
return new PropertyLicenseMetadata
{
    Name = $"رخصة عقار صرح" + {p.PropertyCode},
    Description = $"رخصة ملكية عقارية رقمية صادرة من سجل العقارات الليبي • صاحب الحق",
    Image = $"{verifyUrl}/preview.png",
    ExternalUrl = verifyUrl,
    Attributes = new() { /* Property Code, Type, Region, Area, Decree, Approved At */ },
    Sarh = new SarhExtension
    {
        PropertyId = p.Id,
        PropertyCode = p.PropertyCode,
        OwnerDid = ownerDid,
        DecreeNo = decree,
        ApprovedAt = approvedAt,
        DeedSha256 = p.DeedSignedHash ?? "", // ربط رخصة السلسلة بالصك الموقع
        PolygonGeoJson = polygon, // مضمّنًا في الميتاداتا RFC 7946 مضلع الأرض
        VerifyUrl = verifyUrl,
    },
};
```

٢.٤ تثبيت الميتاداتا على IPFS وتجزئتها

```
var metaJson = JsonSerializer.Serialize(metadata, ...);
var pin = await ipfs.PinJsonAsync(metaJson, ct); // → { IpfsUri = "ipfs://<cid>", Sha256 }
```

في وضع التطوير، `StubIpfsService` يحسب `sha256` للمحتوى، يصوغ CID زائفًا مستقرًا (`[..52] sha256 + bafk`)، ويخزّن الملف تحت دلو `ipfs-stub` على القرص ليعدمه مسار التحقق. الانتقال إلى مثبّت حقيقي (Pinata / web3.storage) = تنفيذ بديل لنفس الواجهة `IIpfsService`.

٢.٥ السكّ على السلسلة عبر IBlockchainService

العقد الموحّد للسكّ — لا يفرّق المستدعي بين Stub وEthereum:

```
// IBlockchainService - الواجهة المجردة
public interface IBlockchainService
{
    string Network { get; }
    string Standard { get; } // "ERC-721"
    string ContractAddress { get; }
    Task<MintReceipt> MintAsync(MintRequest request, CancellationToken ct);
    Task<TransferReceipt> TransferAsync(TransferRequest request, CancellationToken ct);
    Task<string?> OwnerOfAsync(string tokenId, CancellationToken ct); // ownerOf السلسلة
    string ExplorerTxUrl(string txHash);
    string ExplorerTokenUrl(string tokenId);
}
```

نداء السكّ من خدمة الرخص:

```
var receipt = await chain.MintAsync(new MintRequest
{
    PropertyId = property.Id,
    OwnerDid = ownerDid,
    TokenUri = pin.IpfsUri, // tokenURI يشير إلى ميتاداتا IPFS
    MetadataSha256 = pin.Sha256, // المرساة المقاومة للتلاعب
}, ct);
```

التنفيذ الحتمي (تطوير) يولّد tokenId كـ uint256 (سلسلة عشرية) و txHash بطول رمز إيثيريوم حقيقي، مشتقّين من تجزئة propertyId | ownerDid | metadataSha256 — فيبقى الناتج ثابتاً لنفس العقار ويصلح للاختبارات الآلية:


```
// StubBlockchainService.MintAsync - المقتطف الجوهري
var seed = $"{req.PropertyId:N}|{req.OwnerDid}|{req.MetadataSha256}";
var tokenIdBytes = SHA256.HashData(Encoding.UTF8.GetBytes($"token:{seed}"));
var tokenId = new BigInteger(tokenIdBytes, isUnsigned: true, isBigEndian: true).ToString();
var txHash = "0x" + HashHex($"tx:{seed}", 64);
// → MintReceipt { TokenId, ContractAddress, Network, Standard, OwnerDid, OwnerAddress, TxHash,
```

٢.٦ حفظ سجل الملكية في قاعدة البيانات

بعد نجاح السكّ، يُكتب صفّ الرخصة + الصفّ الأول من سلسلة الحيازة، ثم تُحدّث حالة العقار، كلّها في حفظ واحد:

```

db.PropertyNfts.Add(new PropertyNft {
    PropertyId = property.Id, TokenId = receipt.TokenId,
    ContractAddress = receipt.ContractAddress, Network = receipt.Network,
    Standard = receipt.Standard, OwnerDid = receipt.OwnerDid, OwnerAddress = receipt.OwnerAddress,
    MetadataUri = pin.IpfsUri, MetadataSha256 = pin.Sha256,
    MintTxHash = receipt.TxHash, MintBlockNumber = receipt.BlockNumber,
    MintedByOfficerId = actor.OfficerId, MintedAt = receipt.MintedAt, Status = "minted",
});

db.OwnershipHistory.Add(new OwnershipHistory {
    PropertyId = property.Id, NftId = nft.Id,
    FromDid = null, ToDid = ownerDid, // أول صف: من = لا أحد
    FromCitizenId = null, ToCitizenId = owner.Id,
    TransferTxHash = receipt.TxHash, TransferBlockNumber = receipt.BlockNumber,
    Reason = "initial_mint", RecordedByOfficerId = actor.OfficerId,
    TransferredAt = receipt.MintedAt,
});

property.Status = "minted";
property.ApprovedByManagerId = actor.OfficerId;
property.FinalApprovedAt = finalApprovedAt;
await db.SaveChangesAsync(ct);

```

ملاحظة تصميمية صريحة في الكود: الملكية القانونية تبقى مُتتَبَعَة عبر `properties.owner_citizen_id` ؛ الـ NFT مرآة قابلة للتحقق وليس بديلاً قانونياً. أيّ تغيير في المالك يجب أن يُرفق بصفّ في `ownership_history` (مفروض في طبقة الخدمة).

٢.٧ اشتقاق هوية المالك (DID)

```
private static string OwnerDidFor(Citizen c)
{
    // DID ssi_wallets. مؤقت مستقر مشتق من لاحقة معرف المواطن، إلى أن يُربط جدول DID
    var hex = c.Id.ToString("N");
    return $"did:sov:LY:{hex[^16..]}";
}
```

تُقرن دائماً الـ DID (هوية W3C خارج السلسلة) بعنوان على السلسلة (owner_address)، فتبقى عمليات التحقق بالهوية الذاتية (SSI) صالحة حتى بعد تدوير المحفظة.

٢.٨ نقل الملكية لاحقاً (TransferService)

بيع/إرث/هبة/قرار محكمة/تصحيح إداري ينقذها POST /api/v1/property-nfts/{id}/transfer :
نداء chain.TransferAsync ثم تحديث ثلاثة مواضع في حفظ واحد —
property_nfts (المالك الجديد + حالة transferred)، و ownership_history (صفّ جديد)،
و properties.owner_citizen_id (المالك القانوني). تُفرض نفس البوابة الأمنية (لا نقل لعقار
عليه نزاع نشيط) ويُطلب سبب صالح + ملاحظة إلزامية لقرار المحكمة/التصحيح.

٢.٩ التحقق العام (ownerOf + verify على السلسلة)

GET /api/v1/verify/{code} يقرأ بيانات العقار + تجزئة التحقق + الرمز، ثم يستعلم
chain.OwnerOfAsync(tokenId) ليُظهر المالك الحيّ على السلسلة الذي قد يختلف عن السجل
بعد

نقل خارجه — فيُكشف أي انحراف للعموم.

٢.١٠ المسارات (REST) وتسجيل الخدمات (DI)

الخدمة	المسار	الفعل
LicenseService.FinalApproveAsync (السكّ)	/api/v1/properties/{id}/final-approve	POST
سكّ جماعي	/api/v1/properties/bulk-final-approve	POST
TransferService.TransferAsync (النقل)	/api/v1/property-nfts/{id}/transfer	POST
NftsService.ListAsync (سجلّ الرخص)	/api/v1/property-nfts	GET
NftsService.ListHistoryAsync (سلسلة الحيازة)	/api/v1/property-nfts/{id}/history	GET
NftsService.ListMyAsync (رخص المواطن)	/api/v1/me/nfts	GET

```
// Program.cs — Stub افتراضيًا IPFS حقن طبقة السلسلة و
builder.Services.Configure<BlockchainOptions>(builder.Configuration.GetSection(BlockchainOptions
builder.Services.Configure<IpfsOptions>(builder.Configuration.GetSection(IpfsOptions.SectionName
builder.Services.AddSingleton<IBlockchainService, StubBlockchainService>();
builder.Services.AddSingleton<IIpfsService, StubIpfsService>();
```

٢.١١ حالة التنفيذ: Stub مقابل Ethereum — وكيف نتنقل للإنتاج

التصميم يعزل «ماذا» (سكّ/نقل/قراءة مالك) عن «كيف» (سلسلة فعلية أم بديل). للانتقال للإنتاج على إيثريوم (موتّق حرفيًا في تعليقات BlockchainOptions.cs):

1. ضبط `Sarh:Blockchain:Mode = "ethereum"`.

2. تعبئة `RpcUrl` Infura/Alchemy أو عقدة Anvil/Hardhat محلية و `ContractAddress`

و `MinterPrivateKeyEnc` (مفتاح السكّ مُعَمّى بـ KMS — لا يُخزّن صريحًا أبدًا).

1. إضافة حزمة `Nethereum.Web3` وتنفيذ `EthereumBlockchainService`

`IBlockchainService`

(ينادي دوال العقد `ownerOf` / `safeTransferFrom` / `mint`).

1. تبديل سطر التسجيل في `Program.cs` إلى التنفيذ الجديد — لا يتغير أي شيء آخر في

LicenseService أو TransferService أو قاعدة البيانات أو الواجهة.

الشبكات المسموح بها في ck_nft_network جاهزة مسبقًا:

ethereum-mainnet , ethereum-sepolia , polygon-mainnet , polygon-amoy , hyperledger-fabric

٣. نموذج بيانات البلوكتشين (الترحيل 028)

```

-- property_nfts: الرابط (نظام التسجيل) بين العقار وهويته على السلسلة
CREATE TABLE property_nfts (
    id                UNIQUEIDENTIFIER PRIMARY KEY DEFAULT NEWID(),
    property_id       UNIQUEIDENTIFIER NOT NULL REFERENCES properties(id),
    token_id          NVARCHAR(80) NOT NULL,           -- uint256 (تفادي الفيض) كنص
    contract_address  NVARCHAR(80) NOT NULL,
    network           NVARCHAR(40) NOT NULL CHECK (network IN (...)),
    standard          NVARCHAR(24) NOT NULL DEFAULT N'ERC-721',
    owner_did         NVARCHAR(160) NOT NULL,          -- خارج السلسلة W3C هوية
    owner_address     NVARCHAR(80) NULL,              -- العنوان على السلسلة
    metadata_uri      NVARCHAR(255) NOT NULL,          -- ipfs://<cid>
    metadata_sha256   CHAR(64) NOT NULL,              -- المرساة المقاومة للتلاعب
    mint_tx_hash      NVARCHAR(80) NOT NULL,
    mint_block_number BIGINT NULL,
    status            NVARCHAR(16) NOT NULL CHECK (status IN
        (N'pending', N'minted', N'transferred', N'burned', N'failed')),
    ...
);
-- رمز واحد لكل عقار (فهرس فريد مُرشح على الحالات النشطة)
CREATE UNIQUE INDEX ux_nft_property ON property_nfts(property_id)
    WHERE status IN (N'pending', N'minted', N'transferred');

-- ownership_history: (مثل audit_log) سلسلة حياة للإضافة فقط
CREATE TRIGGER tr_ownership_history_no_update ON ownership_history
INSTEAD OF UPDATE AS BEGIN RAISERROR(N'append-only; UPDATE not permitted.',16,1); END
CREATE TRIGGER tr_ownership_history_no_delete ON ownership_history
INSTEAD OF DELETE AS BEGIN RAISERROR(N'append-only; DELETE not permitted.',16,1); END

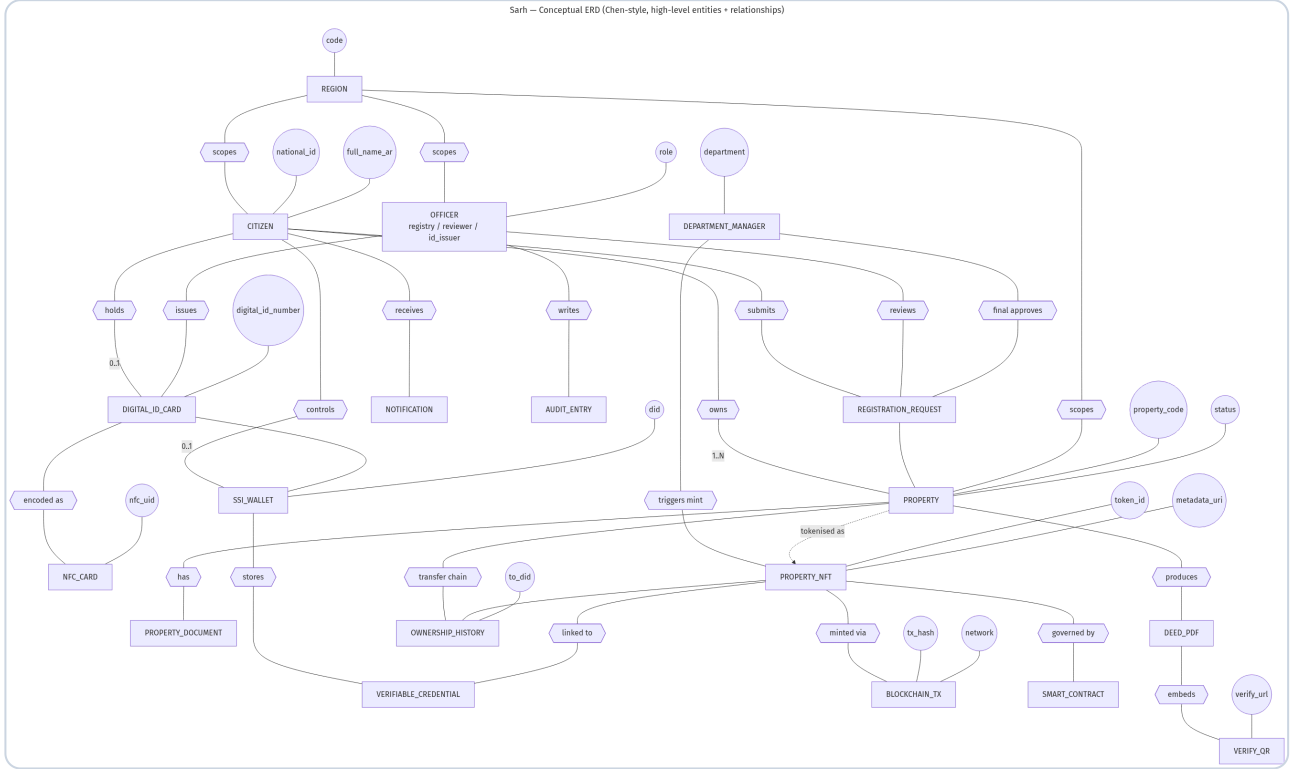
```

النقاط التي يُتصحّ المراجع بالتدقيق فيها: (أ) فصل الهوية القانونية عن مرآة السلسلة؛
 (ب) الإضافة-فقط على سلسلة الحياة بمشغلات (لا بالاتفاق فقط)؛ (ج) المرساة بالتجزئة
 (metadata_sha256) لمقاومة التلاعب؛ (د) فريدة الرمز لكل عقار عبر فهرس مُرشح.

٤. المخطط المفاهيمي (Conceptual ERD)

نظرة عالية المستوى (نمط Chen) للكيانات والعلاقات. كيانات البلوكتشين مُبرزة:

PROPERTY_NFT ، OWNERSHIP_HISTORY ، BLOCKCHAIN_TX ، SMART_CONTRACT

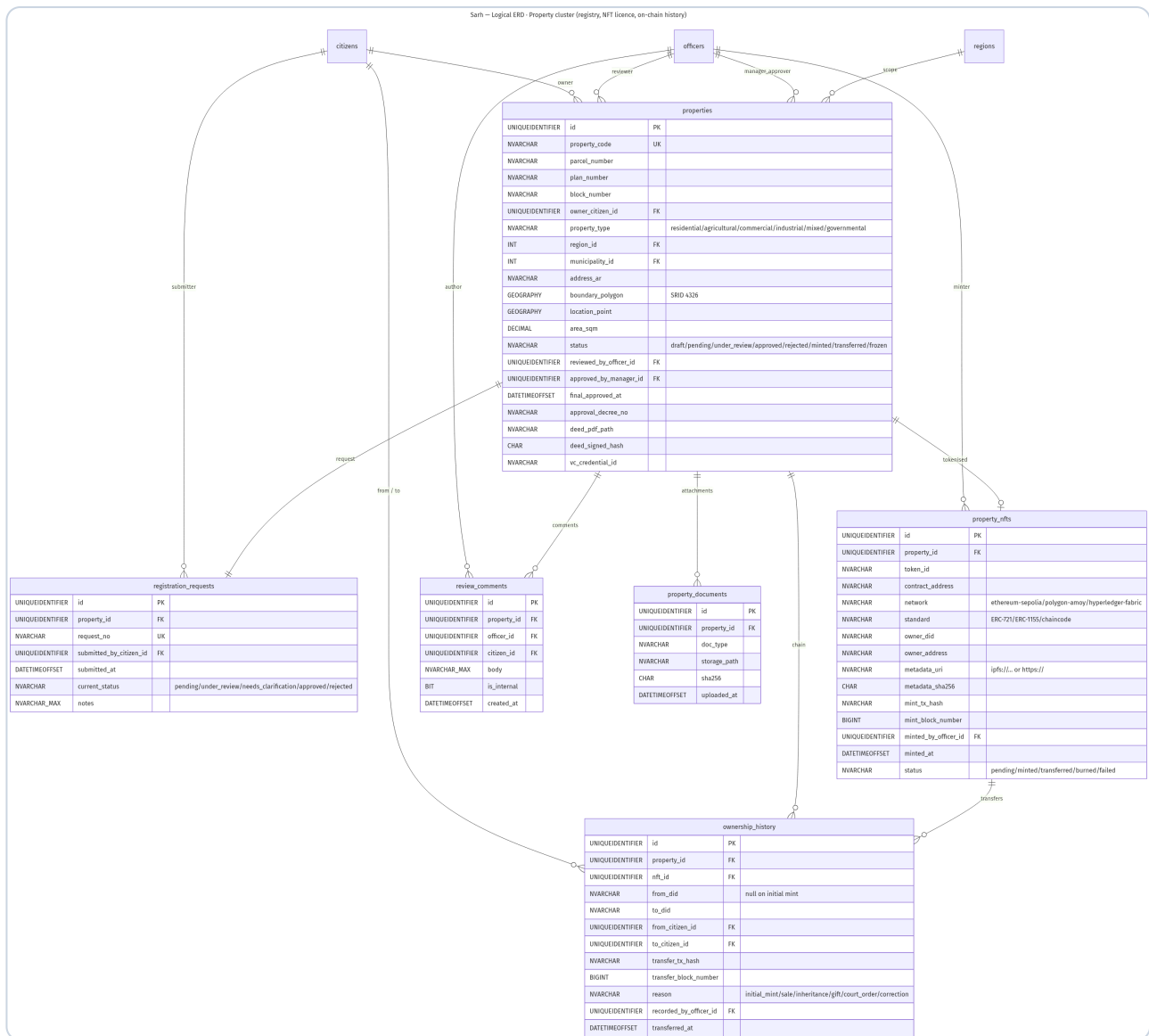


المخطط المفاهيمي — المصدر: docs/diagrams/conceptual-erd.mmd

علاقات البلوكتشين في هذا المخطط: DEPARTMENT_MANAGER -(triggers mint)→ PROPERTY_NFT ،
PROPERTY -(tokenised as)→ PROPERTY_NFT و PROPERTY_NFT -(minted via)→ BLOCKCHAIN_TX ،
PROPERTY_NFT -(governed by)→ SMART_CONTRACT و PROPERTY_NFT → OWNERSHIP_HISTORY ،
(سلسلة النقل)، و PROPERTY_NFT -(linked to)→ VERIFIABLE_CREDENTIAL .

٥. المخطط المنطقي (Logical ERD / UML)

المخطط الكامل بالجدول والمفاتيح (PK/FK/UQ). نظراً لحجمه، يُصَيَّر مقسماً إلى ثلاث لوحات (الهوية / العقارات / النظام) — مصدرها الموحد docs/diagrams/db-schema.mmd . اللوحة الخاصة بالعقارات أدناه تُظهر **جدولي البلوكتشين** property_nfts و ownership_history بكامل أعمدهما وعلاقتهما بـ properties و citizens و officers .



المخطط المنطقي — لوحة العقارات والبلوكشين (property_nfts + ownership_history). المصدر:

[docs/diagrams/db-schema.mmd](#)

اللوحتان الأخريان: الهوية الرقمية [docs/diagrams/db-schema-identity.png](#) ، والنظام/
التدقيق

[docs/diagrams/db-schema-system.png](#) . والمخطط المنطقي الكامل بصيغة نصية في db-
schema.mmd

العلاقات الجوهرية للبلوكشين على المستوى المنطقي:

- **properties ||--o| property_nfts** — لكل عقار رمز واحد كحد أقصى («tokenised»).
- **property_nfts ||--o{ ownership_history** — لكل رمز سلسلة تحويلات.
- **officers ||--o{ property_nfts** — الموظف الساك («minter»).
- **citizens ||--o{ ownership_history** — طرفا التحويل (from / to).

٦. مخطط الفئات (Class Diagram)

نطاق الـ NET 8 + طبقة البلوكتشين. الفئات ذات الصلة:

، BlockchainService ، SmartContract («solidity / chaincode»)، IpfsService
، DepartmentManager.TriggerNftMint(...) إضافةً إلى ، PropertyNft ، OwnershipHistory

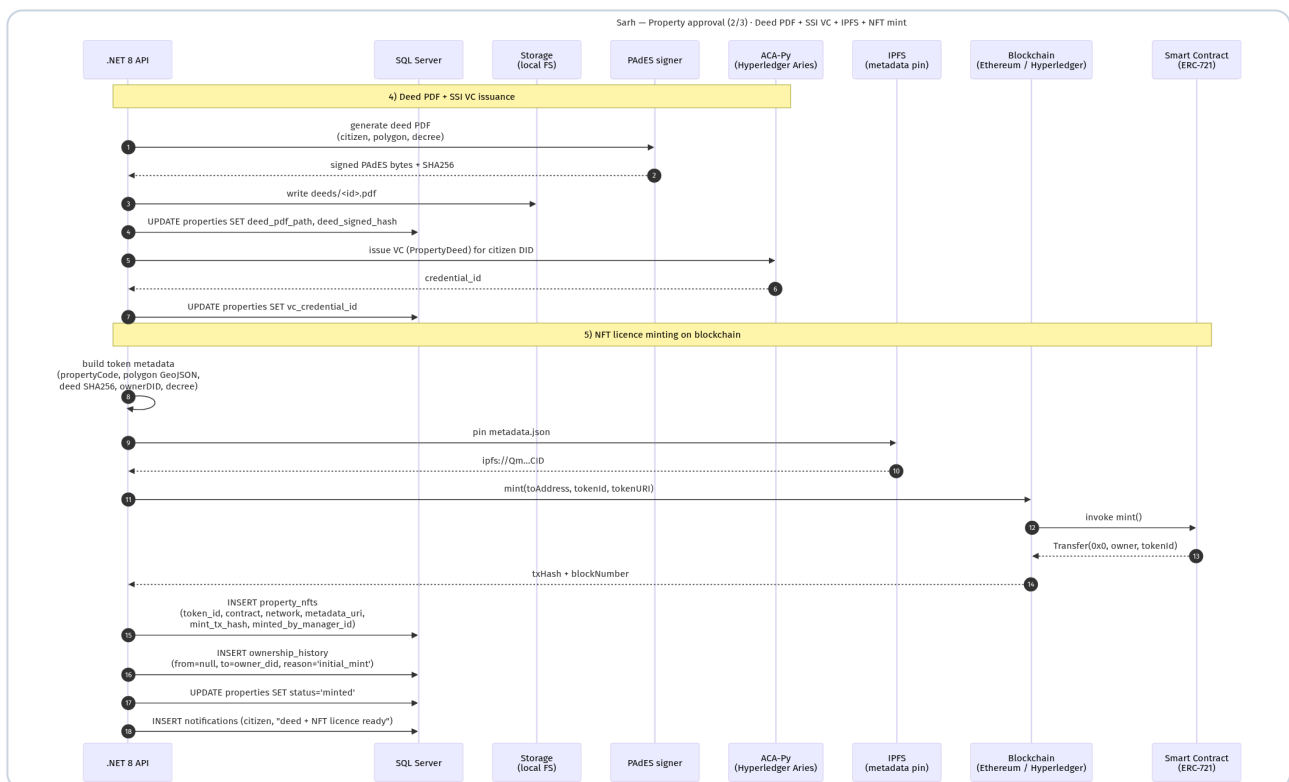


صَرَح . مراجعة البلوك تشين والمخططات . 20 / 18

فئة المخطط	الكود الفعلي
BlockchainService	واجهة IBlockchainService + تنفيذ StubBlockchainService (و EthereumBlockchainService مستقبلاً).
IpfsService	واجهة IIpfsService + تنفيذ StubIpfsService.
SmartContract	عقد ERC-721 الخارجي (Solidity) / Chaincode — تُنادى دواله .mint/safeTransferFrom/ownerOf.
PropertyNft / OwnershipHistory	كيانات EF Core على الجدولين.
DepartmentManager.TriggerNftMint	.LicenseService.FinalApproveAsync

٧. مخطط التسلسل لعملية السكّ (للسياق)

يوضّح المرحلتين ٤ و٥: إصدار الصكّ PAdES + اعتماد VC، ثم تثبيت الميادات على IPFS والسكّ على السلسلة وكتابة .property_nfts + ownership_history.



تسلسل السكّ — المصدر: docs/diagrams/sequence-property-mint.mmd

التسلسل الكامل من التقديم حتى التحقق متاح في `docs/diagrams/sequence-property-`
`approval.mmd`

٨. خلاصة للمراجع

- البلوكتشين ليس ناقصًا: الكود ونموذج البيانات والمسارات والمخططات كلها موجودة (الأقسام ٦-١).
- اعتماد الأرض على السلسلة = سكّ رخصة **ERC-721** عبر `LicenseService.FinalApproveAsync`
- مرساة بميتاداتا IPFS مجزأة، ومسجلة في `ownership_history` + `property_nfts`.
- التصميم قابل للإنتاج: طبقة سلسلة مجزأة (Stub اليوم، Ethereum/Nethereum بتبديل إعداد)،
مع فصل الملكية القانونية عن مرآة السلسلة، وسلسلة حيازة للإضافة-فقط، ومرساة مقاومة للتلاعب.

ملفات مرجعية: كود السلسلة `apps/api-dotnet/Blockchain/*` — خدمات السكّ/النقل
— `apps/api-dotnet/Workflow/{LicenseService,TransferService,NftsService}.cs`
المخطط
— `infra/mssql/migrations/028_property_nfts_ownership_history.sql` — الرسوم
`docs/diagrams/`